MINI PROJECT REPORT

PLANT DISEASE DETECTION USING DEEP LEARNING

A Computer Vision Approach for Early Identification of Plant Leaf Diseases

Submitted by

Name: DHANYA MV

Register Number: 711524BCS035

Class & Section: III- CSE- B

Department of Computer Science and Engineering

ABSTRACT

Agriculture plays a major role in food security and economic development. Plant diseases can reduce crop quality and yield, particularly when symptoms are not identified at an early stage. Traditional disease identification relies heavily on visual inspection by farmers or agricultural experts. This process may be time-consuming, subjective, and difficult to scale across large farms. Plant Disease Detection Using Deep Learning proposes an image-based system that analyses photographs of plant leaves and predicts whether a leaf is healthy or affected by a disease.

The proposed approach uses deep learning, particularly a Convolutional Neural Network (CNN), to learn visual patterns such as spots, discoloration, lesions, and changes in leaf texture. Images are collected from a labelled dataset, resized and normalized, and divided into training, validation, and testing sets. A CNN model is trained to classify the images into predefined plant-health categories. The model can then be integrated into a simple web or mobile interface where a user uploads a leaf image and receives a predicted class with a confidence score.

The project is intended as a decision-support tool rather than a replacement for agricultural experts. Its reliability depends on dataset quality, image conditions, and the similarity between training images and real farm conditions. Evaluation can be performed using accuracy, precision, recall, F1-score, and a confusion matrix. The system may support faster preliminary screening, awareness, and timely consultation, while further field testing is required before practical deployment.

KEYWORDS

Deep Learning, Plant Disease Detection, Convolutional Neural Network, Image Classification, Computer Vision, Agriculture, Leaf Image, Transfer Learning.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Existing System
6. Proposed System
7. System Requirements
8. Dataset Description and Preparation
9. Methodology
10. System Architecture
11. CNN Model Design
12. Implementation Plan

13. Evaluation Metrics
14. Expected Results
15. Advantages
16. Limitations
17. Applications
18. Future Enhancements
19. Conclusion
20. References

INTRODUCTION

Plants are vulnerable to diseases caused by fungi, bacteria, viruses, and other biological or environmental factors. Symptoms may appear on leaves as yellowing, brown or black spots, powdery growth, curling, wilting, or unusual patterns. If a disease is not noticed promptly, it may spread within a crop and lead to reduced productivity. Accurate identification is therefore an important part of crop management.

In conventional practice, farmers often inspect plants visually or seek advice from agricultural specialists. Although expert inspection is valuable, access to experts may be limited in some areas. The result can also depend on the observer's experience, lighting, crop variety, and stage of infection. Computer vision offers a way to analyse plant images consistently, and deep learning can automatically learn useful features from examples rather than relying entirely on manually designed rules.

A deep-learning-based plant disease detection system accepts an image of a leaf, processes the image, and predicts a class learned during training. For example, a dataset may contain healthy leaves and leaves affected by selected diseases in tomato, potato, or pepper plants. The exact classes depend on the dataset used. The output is a prediction, not a guaranteed diagnosis; unusual images should be reviewed by a qualified agricultural professional.

PROBLEM STATEMENT

Manual plant disease identification can be slow, difficult to scale, and inconsistent when many plants must be inspected. Farmers may not always have immediate access to specialists, and visible symptoms of different diseases can look similar. The problem addressed by this project is to develop an image-classification system that uses a labelled collection of plant-leaf images to predict selected disease categories accurately and efficiently. The system should provide a clear result, be straightforward to use, and be evaluated on images that were not used during training.

OBJECTIVES

- To study the use of deep learning and computer vision for plant disease identification.
- To prepare a labelled dataset of healthy and diseased plant-leaf images.
- To preprocess images so that they have a consistent size and numerical representation.
- To design and train a CNN or apply transfer learning using a pretrained image model.
- To classify uploaded leaf images into the supported plant-health categories.
- To evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- To create a simple user interface for uploading an image and viewing the predicted class.
- To document limitations and encourage expert verification for important agricultural decisions.

SCOPE OF THE PROJECT

The mini-project focuses on classifying plant-leaf images into a finite set of categories represented in the training dataset. It may begin with a small number of classes, such as healthy leaves and selected diseases of one crop, before being extended to multiple crops. The system can be developed as a Python prototype and connected to a lightweight web interface. It does not independently confirm the pathogen, recommend chemical treatment, estimate crop loss, or replace laboratory testing and expert diagnosis.

The system's performance should be tested on both clean dataset images and, where possible, photographs taken under practical conditions. The scope should clearly state which crops and diseases are supported, because a model cannot reliably identify classes it has not learned.

EXISTING SYSTEM

Existing approaches include visual inspection by farmers, consultation with agricultural officers, laboratory diagnosis, and image-processing systems that use manually designed features. Human expertise can consider contextual details and remains essential for difficult cases. However, manual inspection may require time and experience. Traditional computer-vision methods often depend on colour thresholds, segmentation, shape measurements, or texture descriptors, which may be sensitive to lighting and background changes.

Drawbacks of the Existing System

- Visual assessment may vary between observers and may be difficult to scale.
- Access to expert support may be limited in remote farming areas.
- Manual feature extraction may not generalize well across different image conditions.
- Some diseases exhibit similar visual symptoms, making classification challenging.
- A system trained on laboratory images may not perform equally well on field photographs.

PROPOSED SYSTEM

The proposed system uses a CNN-based image classifier to learn features directly from plant-leaf images. During training, the model receives images with known labels and adjusts its parameters to

reduce classification error. During prediction, a new image is passed through the same preprocessing pipeline, and the trained model returns class scores. The class with the highest score is displayed along with a confidence estimate.

A typical workflow includes image upload, validation, preprocessing, model inference, result display, and optional feedback collection. Transfer learning may be used when the dataset is small: a model pretrained on a large image collection is adapted to the plant-disease classes. This can reduce training time and improve performance, but the final results must still be measured on a separate test set.

Main Modules

- Image Acquisition: accepts leaf images from a dataset or user upload.
- Preprocessing: checks image format, resizes it, converts it to an array, and applies the required normalization.
- Model Training: trains a CNN or fine-tunes a pretrained model using labelled examples.
- Prediction: generates scores for each supported class and selects the highest-scoring class.
- Result Display: presents the predicted class and, if appropriate, the model's confidence score.
- Evaluation: compares predictions with the true labels and calculates performance metrics.

SYSTEM REQUIREMENTS

Hardware Requirements

- Computer or laptop with a modern multi-core processor.
- At least 8 GB RAM recommended; more memory is helpful for larger datasets.
- Adequate storage for the dataset, source code, and trained model.
- GPU optional; a compatible GPU can reduce training time for larger models.

Software Requirements

- Operating System: Windows, Linux, or macOS.
- Programming Language: Python 3.
- Libraries: TensorFlow/Keras or PyTorch, NumPy, Matplotlib, scikit-learn, and Pillow or OpenCV.
- Development Environment: Visual Studio Code, Jupyter Notebook, or Google Colab.
- Optional Interface: Flask or Streamlit for a simple web application.

DATASET DESCRIPTION AND PREPARATION

A labelled dataset is required to train and evaluate the model. One commonly used research dataset is PlantVillage, which contains images of healthy and diseased leaves from several crop types. Dataset selection must be based on the project's target crops, class labels, licence, and suitability for the intended use. A dataset should not be described as representing all real-world field conditions unless that has been verified.

Dataset Organization

A convenient directory structure is one folder per class, with images stored inside the corresponding folder. For example, a small project may use folders named Healthy, Tomato_Early_Blight, and Tomato_Late_Blight. The class names should match the actual dataset labels and must not be invented after training.

Data Splitting

Divide the dataset into training, validation, and testing subsets. A possible split is 70% training, 15% validation, and 15% testing, although other ratios may be appropriate. The training set updates model parameters, the validation set helps tune hyperparameters and monitor overfitting, and the test set is reserved for final evaluation. Near-duplicate images should not be distributed across different subsets, because this can inflate measured performance.

Image Preprocessing

- Resize all images to a fixed input shape, for example 224 × 224 pixels, depending on the selected model.
- Convert images to RGB if required by the model.
- Normalize pixel values according to the model's expected input range.
- Check for unreadable files, incorrect labels, duplicates, and severe class imbalance.
- Apply moderate data augmentation only to training images, such as small rotations, flips, or zooms, when these transformations preserve the meaning of the image.

METHODOLOGY

Image Collection

Collect images from a public dataset or a carefully labelled image collection. Record the dataset source, version, class list, and any usage conditions. Include enough examples per class to support learning and avoid relying on a very small number of images.

Preprocessing and Augmentation

Images are resized and converted into numeric tensors. Pixel normalization improves compatibility with the selected model. Data augmentation creates varied training examples and can reduce overfitting, but it should not introduce unrealistic patterns or change the disease label.

Model Selection

A CNN is suitable because convolutional filters learn local visual patterns, including edges, textures, and lesion shapes. For a beginner-level project, a small CNN can be built from convolution, activation, pooling, flattening or global average pooling, and dense classification layers. Alternatively, transfer learning can use a pretrained model such as MobileNetV2 or EfficientNet, followed by fine-tuning on the selected plant-disease dataset.

Training

During training, the model predicts a class distribution for each image. A loss function, commonly categorical cross-entropy or sparse categorical cross-entropy, measures the difference between predicted scores and true labels. An optimizer such as Adam updates model parameters through backpropagation. Training should be monitored using validation loss and validation accuracy. Early stopping and model checkpointing can help retain the best validation model.

Prediction

For a new image, the application performs the same resizing and normalization used during training. The model generates a score for every supported class, and the highest-scoring class is selected. A confidence score should be presented carefully: a high softmax score does not necessarily mean the prediction is correct, especially for unfamiliar images or classes outside the training data.

Output

The application displays the uploaded image, predicted class, and optional confidence score. A responsible implementation should also show a message that the output is an automated prediction and that uncertain or high-impact cases should be checked by an agricultural expert.

SYSTEM ARCHITECTURE

The high-level architecture consists of the following stages:

1. User or Dataset → Plant Leaf Image
2. Image Validation and Preprocessing → Resized, Normalized Image
3. Trained CNN / Transfer-Learning Model → Class Scores
4. Post-processing → Predicted Disease or Healthy Class
5. User Interface → Prediction, Confidence Estimate, and Advisory Disclaimer

The training pipeline is separate from the prediction pipeline. Training uses labelled images and validation feedback to create a saved model file. The prediction application loads that saved model and uses it to classify new images. This separation makes it easier to update the model without redesigning the interface.

CNN MODEL DESIGN

A basic CNN may contain the following conceptual layers. The exact number of filters and layers should be selected through experiments rather than treated as a universal configuration.

- Input Layer: accepts a fixed-size RGB image, such as 224 × 224 × 3.
- Convolution Layer: applies learnable filters to detect local visual features.
- Activation Layer: ReLU introduces non-linearity so the model can learn complex patterns.
- Pooling Layer: reduces spatial dimensions and computational cost.
- Repeated Convolution Blocks: learn increasingly complex patterns.

- Global Average Pooling or Flatten: converts feature maps into a representation for classification.
- Dense Layer: combines learned features for the final decision.
- Output Layer: uses softmax for a single-label, multi-class classification task.

For a dataset with K classes, the output layer typically contains K units. During training, labels and outputs must use compatible formats. If transfer learning is used, the model's input preprocessing requirements must be followed precisely.

IMPLEMENTATION PLAN

The project can be implemented in phases so that each part can be tested independently.

6. Phase 1 — Define the crop types, disease classes, dataset, and evaluation plan.
7. Phase 2 — Download or organize the labelled dataset and inspect sample images.
8. Phase 3 — Split the data and build the preprocessing pipeline.
9. Phase 4 — Train a baseline CNN or transfer-learning model.
10. Phase 5 — Evaluate the model on the validation and test sets and inspect errors.
11. Phase 6 — Save the trained model and create an image-upload interface.
12. Phase 7 — Test the full workflow with unseen images and document limitations.

Example Pseudocode

```
Load labelled plant-leaf dataset
Split data into training, validation, and testing sets
Create image preprocessing pipeline
Build CNN or load pretrained image model
Train model using training images
Monitor validation metrics and save the best model
Evaluate saved model on the held-out test set
For each uploaded leaf image:
    Validate image file
    Resize and normalize image
    Predict class scores using saved model
    Display predicted class and suitable disclaimer
```

EVALUATION METRICS

The model should be evaluated on a held-out test set. Accuracy alone can be misleading when classes are imbalanced, so class-wise metrics and a confusion matrix should also be reported.

Accuracy

Accuracy = (Number of correct predictions / Total number of predictions) × 100.

Precision

Precision = TP / (TP + FP). Precision indicates how many predictions for a class were correct. Here, TP is true positives and FP is false positives.

Recall

Recall = TP / (TP + FN). Recall indicates how many actual examples of a class were correctly identified. FN means false negatives.

F1-Score

F1-score = 2 × (Precision × Recall) / (Precision + Recall). It balances precision and recall. Macro-averaged metrics give each class equal weight, while weighted averages account for class frequencies.

Confusion Matrix

A confusion matrix compares actual classes with predicted classes. It helps identify which diseases the model confuses with one another. For example, two diseases with similar leaf symptoms may be misclassified more frequently.

Reporting Results

Do not insert assumed accuracy values into the final report. Train the model and record the actual test results. A results table may be completed as follows:

Metric	Measured result
Test accuracy	_____
Macro precision	_____
Macro recall	_____
Macro F1-score	_____
Number of test images	_____

EXPECTED RESULTS

The expected outcome is a working prototype that can distinguish the selected healthy and diseased leaf categories represented in its training data. The model should learn visual patterns associated with each class and provide a predicted label for a previously unseen image. Actual performance will depend on dataset size, label quality, class balance, model choice, training configuration, and test-image conditions. Results must be measured experimentally and reported honestly.

The project should also produce an evaluation summary, such as accuracy and F1-score, a confusion matrix, and examples of correct and incorrect predictions. Error analysis is important because it can reveal weaknesses caused by background differences, poor lighting, overlapping symptoms, or under-represented classes.

ADVANTAGES

- Can provide quick preliminary screening of supported leaf categories.
- Learns visual features automatically from training examples.
- Can process many images consistently once trained.

- Can be connected to a simple web or mobile interface.
- Can support agricultural awareness and further expert investigation.
- Transfer learning may reduce training requirements for smaller projects.

LIMITATIONS

- Performance depends on dataset size, diversity, and label accuracy.
- Laboratory-style images may differ substantially from field photographs.
- Similar diseases or nutrient deficiencies can produce similar visible symptoms.
- The model may fail on crops, diseases, or image conditions not represented during training.
- A model confidence score is not a guarantee of correctness.
- The system does not confirm a pathogen or prescribe treatment.
- Poor connectivity, camera quality, and device limitations may affect practical use.

These limitations can be reduced—but not completely removed—through diverse field data, expert-reviewed labels, careful evaluation, uncertainty handling, and periodic monitoring after deployment.

APPLICATIONS

- Preliminary crop-health screening by farmers and agricultural students.
- Educational demonstrations of deep learning and computer vision.
- Research prototypes for agricultural decision-support systems.
- Integration with farm-management platforms, subject to appropriate validation.
- Support for collecting and reviewing crop-health observations.

FUTURE ENHANCEMENTS

- Expand the dataset to cover more crops, disease classes, growth stages, and field conditions.
- Develop a mobile application that can run offline on supported devices.
- Use object detection or segmentation to locate affected regions on a leaf.
- Add out-of-distribution checks to flag images that do not resemble supported classes.
- Collect expert-reviewed field images and monitor performance across locations and seasons.
- Provide multilingual instructions and accessible user-interface design.
- Connect results to verified agricultural guidance, while avoiding automatic treatment prescriptions without expert review.
- Add explainability visualizations, such as Grad-CAM, to show image regions influencing the prediction.

CONCLUSION

Plant Disease Detection Using Deep Learning demonstrates how computer vision can be applied to an agricultural problem. By training a CNN or adapting a pretrained image model, a system can

learn patterns in plant-leaf images and classify them into supported healthy and diseased categories. The project combines dataset preparation, preprocessing, model training, evaluation, and a user-facing prediction interface.

The system can offer a useful first step for screening, but its predictions should not be treated as definitive diagnoses. Reliable deployment requires testing on diverse real-world images, reporting measured performance, identifying failure cases, and involving agricultural experts in important decisions. With these safeguards, the project provides a practical mini-project for learning deep learning while exploring a meaningful application in agriculture.

REFERENCES

13. Hughes, D. P., and Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv:1511.08060. https://arxiv.org/abs/1511.08060
14. Mohanty, S. P., Hughes, D. P., and Salathé, M. (2016). Using Deep Learning for Image-Based Plant Disease Detection. Frontiers in Plant Science, 7, 1419. https://doi.org/10.3389/fpls.2016.01419
15. Goodfellow, I., Bengio, Y., and Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org/
16. TensorFlow. Image classification tutorials and documentation. https://www.tensorflow.org/tutorials/images/classification
17. Keras. Transfer learning and fine-tuning guide. https://keras.io/guides/transfer_learning/
18. PlantVillage dataset and related research resources. Confirm dataset version, access conditions, and licence before use.